



รายงานโครงงานคอมพิวเตอร์

AI-Assisted Testing vs. Automatic Test Case Generation Algorithms:
Benchmark and Test Coverage Evaluation

จัดทำโดย

673380572-9	นายกฤตธี	ศรียุส	Sec.2
673380585-0	นายธนดล	ไชยศิลา	Sec.2
673380417-1	นายพุมเมธ	ชมศรีสวัสดิ์	Sec.2
673380049-4	นายปิยชญานินท์	โทะนะพันธ์	Sec.1

อาจารย์ที่ปรึกษา : ผู้ช่วยศาสตราจารย์ ดร.ชิตสุธา สุ่มเล็ก

รายงานเล่มนี้เป็นส่วนหนึ่งของรายวิชา CP353201 การประกันคุณภาพซอฟต์แวร์

ภาคเรียนที่ 1 ปีการศึกษา 2569

หลักสูตรวิทยาการคอมพิวเตอร์ สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์

มหาวิทยาลัยขอนแก่น

(เดือน ตุลาคม พ.ศ. 2569)

หัวข้อโครงการ	AI-Assisted Testing vs. Automatic Test Case Generation Algorithms: Benchmark and Test Coverage Evaluation
ผู้จัดทำ	นายกฤติธิ์ ศรีไสย์ นายธนดล ไชยศิลา นายพุฒิเมธ ชมศรีสวัสดิ์ นายปิยชญานินท์ โทณะพันธ์
อาจารย์ที่ปรึกษา	ผู้ช่วยศาสตราจารย์ ดร.จิตสุธา สุ่มเล็ก
ปีการศึกษา	2569

บทคัดย่อ

โครงการนี้ศึกษาการสร้าง Unit Test อัตโนมัติสำหรับโครงการ Java ในชุดข้อมูล Defects4J โดยเปรียบเทียบเครื่องมือ 4 ตัว ได้แก่ UTBot, TARDIS, Claude Sonnet 5 และ Gemini เพื่อประเมินความสามารถในการสร้างชุดทดสอบ การตรวจจับข้อบกพร่อง และความครอบคลุมของโค้ด

การทดลองครอบคลุม active bugs ใน Defects4J โดย UTBot และ Gemini ทดลองกับ 17 projects จำนวน 854 bugs, Claude มีผล 849 bugs และ TARDIS มีผลครบ 854 bugs สำหรับการตรวจจับบั๊ก ใช้หลักว่าชุดทดสอบต้อง fail บน buggy version และ pass บน fixed version

ผลการทดลองพบว่า UTBot ตรวจพบ 7 bugs จาก 854 bugs และเมื่อรวมกับ developer tests ทำให้ line coverage เพิ่มขึ้นจาก 89.25% เป็น 92.41% และ condition coverage เพิ่มขึ้นจาก 83.27% เป็น 87.17% Gemini ตรวจพบ 20 bugs จาก 854 bugs และมี average line coverage 91.33% ขณะที่ Claude ตรวจพบ 46 bugs จาก 849 bugs และมี average line coverage 81.14% ส่วน TARDIS บันทึกการตรวจพบ 63 bugs จาก 854 bugs และมี average line coverage 15.60% ในกลุ่มที่วัดผลได้

โดยสรุป ประสิทธิภาพของเครื่องมือขึ้นอยู่กับความสามารถในการสร้าง test และความเข้ากันได้กับ environment จึงควรประเมินจาก Fault Detection, Code Coverage และความสำเร็จในการ compile/execute ร่วมกัน

คำสำคัญ: Defects4J, Unit Testing, Automated Test Generation, UTBot, TARDIS, Claude, Gemini, Fault Detection, Code Coverage

คำนำ

รายงานฉบับนี้จัดทำขึ้นเป็นส่วนหนึ่งของรายวิชา CP353201 Software Quality Assurance เพื่อศึกษาและเปรียบเทียบการสร้างชุดทดสอบอัตโนมัติด้วย Automatic Test Case Generation Algorithms และ AI-Assisting Tools โดยใช้โครงการ Java จากชุดข้อมูล Defects4J เป็นกรณีศึกษา

ในการทดลอง กลุ่มผู้จัดทำได้เลือกใช้ UTBot และ TARDIS เป็นเครื่องมือสร้างชุดทดสอบอัตโนมัติ และเลือกใช้ Claude Sonnet 5 และ Gemini เป็นเครื่องมือช่วยสร้าง Unit Test ด้วย AI จากนั้นนำชุดทดสอบที่สร้างได้มาประเมินด้านการตรวจจับข้อบกพร่อง ความครอบคลุมของโค้ด และผลการทำงานของแต่ละเครื่องมือ พร้อมทั้งบันทึก Source Code, Test Code, Prompt, Configuration และผลการทดลองไว้ใน GitHub เพื่อให้สามารถศึกษาและทำซ้ำได้

ผู้จัดทำหวังว่ารายงานฉบับนี้จะช่วยให้เห็นถึงความสามารถ ข้อดี และข้อจำกัดของเครื่องมือแต่ละประเภทในการนำมาใช้สำหรับงานทดสอบซอฟต์แวร์ และเป็นแนวทางในการเลือกใช้เครื่องมือให้เหมาะสมกับงานในอนาคต

คณะผู้จัดทำ

สารบัญ

บทคัดย่อ	ก
คำนำ	ข
สารบัญ	ค
บทที่ 1 บทนำ	
1. ที่มา และความสำคัญ	1
2. วัตถุประสงค์	1
3. คำถามการทดลอง	2
บทที่ 2 เครื่องมือ และชุดข้อมูลที่ใช้	
1. ชุดข้อมูล Defects4J	3
2. ตัวชี้วัดที่ใช้ประเมินผล	4
บทที่ 3 วิธีดำเนินการทดลอง	
1. Workflow กลาง	6
2. หลักการเปรียบเทียบ	6
3. การทดลองด้วย UTBot	7
4. การทดลองด้วย TARDIS	9
5. การทดลองด้วย Claude Sonnet 5	11
6. การทดลองด้วย Gemini	12
บทที่ 4 ผลการทดลอง และการเปรียบเทียบ	
1. ภาพรวมผลลัพธ์	14
2. การตีความ Fault Detection	16
3. การตีความ Coverage	16

บทที่ 5 การวิเคราะห์ผล และข้อจำกัด

- | | |
|--|----|
| 1. Automated generation vs AI-assisted generation | 17 |
| 2. Coverage กับ Fault Detection ไม่ใช่สิ่งเดียวกัน | 17 |
| 3. ความสำคัญของ Executability | 17 |
| 4. ความน่าเชื่อถือของ comparison | 17 |
| 5. ปัญหา ข้อจำกัด และ Reproducibility | 17 |

บทที่ 6 สรุปผล

สารบัญภาพ

ภาพที่ 1	ผล weighted coverage ของ UTBot experiment	8
ภาพที่ 2	จำนวน defect ที่แต่ละ pipeline บันทึกว่า detect/kill ได้	15
ภาพที่ 3	Fault-detection rate ตาม denominator ที่แต่ละ pipeline รายงาน	15
ภาพที่ 4	Line coverage ที่รายงาน	16

สารบัญตาราง

ตารางที่ 1 เครื่องมือที่ใช้ในการทดลองและบทบาทของแต่ละเครื่องมือ	3
ตารางที่ 2 จำนวน Active Bugs ของแต่ละโครงการใน Defects4J	3
ตารางที่ 3 Environment และ Version ที่ใช้ในการทดลอง UTBot	7
ตารางที่ 4 ผลการวัด Code Coverage ของ UTBot	8
ตารางที่ 5 Configuration ที่ใช้ในการทดลอง TARDIS	9
ตารางที่ 6 ผลการทดลอง TARDIS แยกรายโครงการ	10
ตารางที่ 7 ผลการทดลองโดยรวมของ Claude Sonnet 5	11
ตารางที่ 8 ผลการทดลอง Claude Sonnet 5 แยกรายโครงการ	11
ตารางที่ 9 ผลการทดลองโดยรวมของ Gemini	12
ตารางที่ 10 ภาพรวมผลการทดลองของเครื่องมือทั้ง 4 ตัว	14
ตารางที่ 11 ปัญหาและข้อจำกัดที่พบในการทดลอง	18
ตารางที่ 12 Artifacts สำหรับการทำการทดลองของแต่ละเครื่องมือ	18

บทที่ 1

บทนำ

1. ที่มา และความสำคัญ

การทดสอบซอฟต์แวร์ระดับหน่วยเป็นพื้นฐานสำคัญของการประกันคุณภาพซอฟต์แวร์ แต่การเขียน test case ด้วยมนุษย์ต้องใช้เวลา และมักขึ้นกับประสบการณ์ของผู้พัฒนา ปัจจุบันมีทั้ง Automatic Test Case Generation Algorithms และ Generative AI ที่สามารถช่วยสร้าง test suite จาก source code ได้ โครงการงานนี้จึงทดลองเครื่องมือจากทั้งสองแนวทางกับ Defects4J ซึ่งรวบรวมข้อบกพร่องจริงของโครงการ Java และมีทั้ง buggy version และ fixed version สำหรับใช้ตรวจสอบความสามารถของ test ในการเปิดเผยข้อบกพร่อง

โจทยรายวิชากำหนดให้เลือก Automatic Test Case Generation 2 ตัว และ AI-Assisting Tools 2 ตัว เพื่อนำมาสร้าง Unit Test สำหรับ Java projects ใน Defects4J พร้อมบันทึกผล วัด test/code coverage, fault detection rate และผลด้านประสิทธิภาพ จากนั้นเปรียบเทียบและจัดเก็บ artifact ทั้งหมดบน GitHub เพื่อให้ผู้อ่านสามารถทำซ้ำการทดลองได้

2. วัตถุประสงค์

- 2.1 ศึกษากระบวนการสร้าง Unit Test แบบอัตโนมัติของ UTBot และ TARDIS
- 2.2 ออกแบบ prompt และ pipeline สำหรับ Claude Sonnet 5 และ Gemini เพื่อสร้าง JUnit 4 tests จาก source code ของ Defects4J
- 2.3 ประเมินความสามารถในการ compile, execute, ตรวจจับ defect และครอบคลุม source code
- 2.4 เปรียบเทียบผลลัพธ์ของเครื่องมือทั้ง 4 โดยคำนึงถึงความแตกต่างของวิธีวัดและข้อจำกัดของแต่ละ pipeline
- 2.5 บันทึก source code, configuration, prompt, generated tests และผลการทดลองบน GitHub เพื่อรองรับการทำซ้ำ

3. คำถามการทดลอง

- 3.1 เครื่องมือแต่ละชนิดสามารถสร้าง test ที่นำไป execute กับ Defects4J ได้มากน้อยเพียงใด
- 3.2 test ที่สร้างสามารถตรวจจับบั๊กจริงจากความแตกต่างระหว่าง buggy และ fixed version ได้หรือไม่
- 3.3 coverage ของ test ที่สร้างเป็นอย่างไร และเมื่อใช้ร่วมกับ developer tests ช่วยเพิ่ม coverage หรือไม่
- 3.4 ข้อจำกัดด้าน compatibility, environment, timeout และ resource ส่งผลต่อผลลัพธ์อย่างไร
- 3.5 metric ไດเหมาะสมต่อการเปรียบเทียบ และ metric ไດไม่ควรเปรียบเทียบโดยตรง

บทที่ 2

เครื่องมือ และชุดข้อมูลที่ใช้

ตารางที่ 1 เครื่องมือที่ใช้ในการทดลองและบทบาทของแต่ละเครื่องมือ

กลุ่ม	เครื่องมือ	บทบาทในการทดลอง
Automatic Test Generation	UTBot	สร้าง Unit Test อัตโนมัติจาก Java bytecode/source context แล้วนำ test ไปทดสอบ buggy/fixed และวัด coverage
	TARDIS	Concolic/Symbolic execution-based generation ผ่าน Docker/JBSE/EvoSuite pipeline พร้อม fallback ระดับ method
AI-Assisting	Claude Sonnet 5	รับ source code + classpath context ผ่าน prompt และสร้าง JUnit 4 tests; temperature = 0
	Gemini	รับ source code + prompt ที่เน้น branch/condition, boundary, null/empty และ invalid states เพื่อสร้าง JUnit 4 tests

1. ชุดข้อมูล Defects4J

Defects4J เป็น benchmark ของข้อบกพร่องจริงในโครงการ Java โดยแต่ละ bug มี buggy version และ fixed version ทำให้สามารถทดสอบ test suite เดียวกันกับทั้งสอง version เพื่อแยกรณีที่ test ล้มเพราะ defect ออกจากกรณีที่ test ล้มเพราะ test เองหรือปัญหา environment ได้ ใน repository ของโครงการพบ active bugs รวม 854 bugs ใน 17 projects

ตารางที่ 2 จำนวน Active Bugs ของแต่ละโครงการใน Defects4J

Project	Bugs	Project	Bugs
Chart	26	JacksonDatabind	110
Cli	39	JacksonXml	6
Closure	174	Jsoup	93
Codec	18	JXPath	22
Collections	28	Lang	61

ตารางที่ 2 จำนวน Active Bugs ของแต่ละโครงการใน Defects4J (ต่อ)

Compress	47	Math	106
Csv	16	Mockito	38
Gson	18	Time	26
JacksonCore	26		

2. ตัวชี้วัดที่ใช้ประเมินผล

2.1 หน่วยการวิเคราะห์

Defects4J bug หนึ่งตัวอาจแก้ไขหลายคลาส ดังนั้น raw result ของบางเครื่องมือมีหลายแถวต่อ bug ในรายงานนี้ใช้ผลระดับ bug เป็นหน่วยหลักสำหรับ Fault Detection และระบุขีดเมื่อ coverage ถูกคำนวณระดับ class หรือเฉลี่ยเฉพาะกลุ่มที่วัดได้ เพื่อหลีกเลี่ยงการตีความค่าร้อยละที่มีตัวหารแตกต่างกันว่าเทียบกันได้โดยตรง

2.2 Fault Detection

สำหรับ UTBot, Claude และ Gemini เกณฑ์ที่เข้มงวดคือ test เดียวกันต้อง fail บน buggy version และ pass บน fixed version จึงนับว่า detect หรือ kill bug ได้จริง สำหรับ TARDIS repository มีทั้ง evaluator source รุ่นที่รองรับ buggy-vs-fixed และไฟล์ผลรวมที่ใช้ schema รุ่นก่อน ดังนั้นค่าที่นำเสนอจาก Coverage_Result_All.csv จะเรียกว่า recorded Fault_Detected และแยกข้อจำกัดนี้ไว้ในการอภิปรายผล

2.3 Code Coverage

- **Line Coverage:** สัดส่วนบรรทัดคำสั่งที่ถูก execute
- **Condition/Branch Coverage:** สัดส่วนเงื่อนไขหรือ branch ที่ถูก execute
- **Weighted Coverage:** รวมจำนวน covered elements / total measurable elements ก่อนคำนวณเปอร์เซ็นต์
- **Average Coverage:** ค่าเฉลี่ยเปอร์เซ็นต์ของหน่วยที่วัดได้ ซึ่งไม่เท่ากับ weighted coverage

2.4 ตัวชี้วัดด้านประสิทธิภาพ

ตัวชี้วัดที่บันทึกใน repository แตกต่างกันตามเครื่องมือ เช่น generation time, test execution time, test-suite LOC, compile rate, จำนวน generated tests และสถานะ timeout/crash การเปรียบเทียบจึงเน้นค่าที่มีนิยามชัดเจนร่วมกัน ส่วนค่าที่มีวิธีเก็บต่างกันจะใช้เพื่ออธิบายพฤติกรรมของเครื่องมือ นั้น ๆ มากกว่าการจัดอันดับ

บทที่ 3

วิธีดำเนินการทดลอง

1. Workflow กลาง

- 1.1 ค้นหา project และ active bug IDs จาก Defects4J
- 1.2 checkout buggy version (<BugID>b) และหา classes.modified
- 1.3 เตรียม source/classpath สำหรับเครื่องมือสร้าง test
- 1.4 สร้าง JUnit test และบันทึก generated test artifact
- 1.5 compile/run test บน buggy version และวัด coverage
- 1.6 เมื่อต้องการตรวจ defect แบบเข้มงวด ให้นำ test เดิมไปรันกับ fixed version (<BugID>f)
- 1.7 บันทึกสถานะ, coverage, failing tests, เวลา และ error
- 1.8 สรุปผลจากระดับ class เป็นระดับ bug และ project

2. หลักการเปรียบเทียบ

เนื่องจากทั้ง 4 pipeline มีความแตกต่างด้าน environment และการ aggregate metric รายงานนี้จึงไม่ใช่ coverage เพียงตัวเดียวเพื่อสรุปว่าเครื่องมือใด 'ดีที่สุด' แต่พิจารณา 4 มิติร่วมกัน ได้แก่

1. coverage
2. evidence ของ fault detection
3. ความสำเร็จในการ compile/execute
4. reproducibility/operational cost

พร้อมระบุ denominator ทุกครั้งที่รายงานอัตรา

3. การทดลองด้วย UTBot

3.1 Environment และการปรับแต่ง

UTBot experiment ถูกพัฒนามาบน Windows 11 โดยใช้ Git Bash สำหรับส่วน Unix-like ของ Defects4J และใช้ Java 11 สำหรับ Defects4J/โปรเจกต์ใน dataset ส่วน Java 17 ใช้ build และ run UTBot CLI นอกจากนี้ใช้ Strawberry Perl, Apache Ant 1.10.18 และ Windows wrapper ของ Major ('major/bin/ant.cmd'). เนื่องจาก Defects4J มีส่วนที่ออกแบบสำหรับ Unix-like environment จึงเก็บ compatibility patch ไว้ใน 'UTBot/patches/defects4j-windows.patch' และ UTBot modifications ไว้ใน 'UTBot/patches/utbot-modifications.patch'

3.2 Reproducible versions

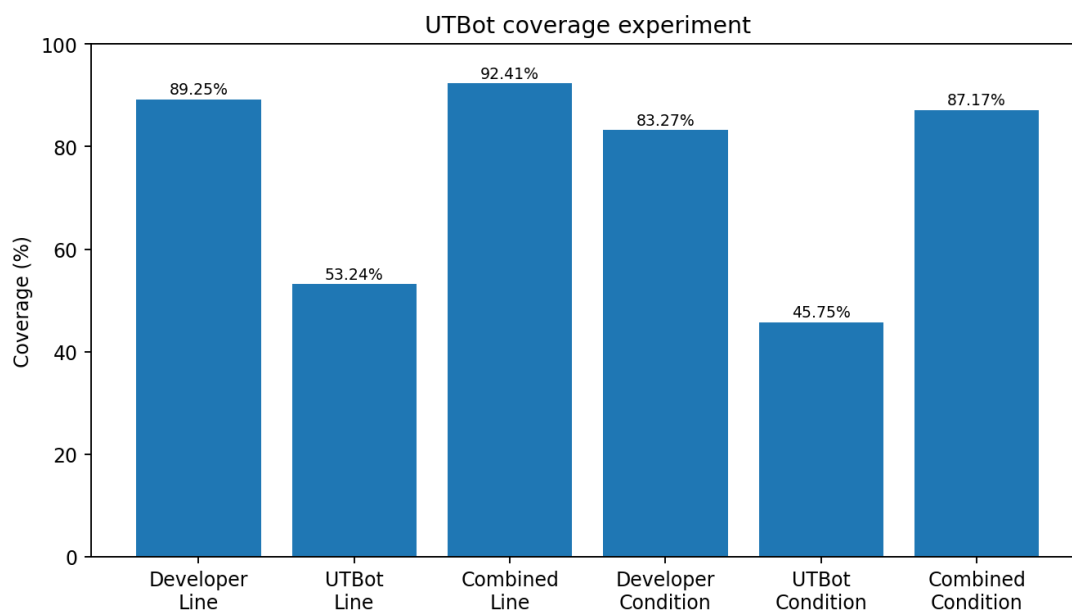
ตารางที่ 3 Environment และ Version ที่ใช้ในการทดลอง UTBot

Component	Version / Commit
Defects4J	8c16da8230843cdc918eaf4ddb449637f02b83c6
UTBotJava	73bd2b2aed09ba94e7cbd875c662f78db10c2da8
Java for Defects4J	OpenJDK 11
Java for UTBot CLI	JDK 17
Apache Ant	1.10.18

3.3 Fault Detection

UTBot ตรวจพบ 7 bugs จาก 854 active bugs ได้แก่ Cli-5, Codec-2, Codec-18, Compress-14, Lang-45, Math-3 และ Math-54 คิดเป็น All-Bug FDR 0.82% ขณะเดียวกันมี 139 bugs ที่ generated tests เข้าสู่ขั้น execute ได้ตามนิยามของ pipeline จึงได้ Executable-Bug FDR 5.04%. การรายงานสองตัวหารช่วยแยกข้อจำกัดด้าน compatibility ออกจากความสามารถในการตรวจ defect เมื่อ test สามารถ execute ได้

3.4 Coverage



ภาพที่ 1 ผล weighted coverage ของ UTBot experiment

จากภาพที่ 1 พบว่า Developer Tests มี Line Coverage เท่ากับ 89.25% และ Condition Coverage เท่ากับ 83.27% ขณะที่ UTBot Tests มีค่า 53.24% และ 45.75% ตามลำดับ อย่างไรก็ตาม เมื่อนำ UTBot Tests มารวมกับ Developer Tests ค่า Coverage เพิ่มขึ้นเป็น 92.41% สำหรับ Line Coverage และ 87.17% สำหรับ Condition Coverage แสดงให้เห็นว่า UTBot สามารถช่วยครอบคลุมส่วนของโค้ดที่ Developer Tests เดิมยังทดสอบไม่ถึงได้

ตารางที่ 4 ผลการวัด Code Coverage ของ UTBot

Metric	Developer	UTBot	Combined	Gain
Line Coverage	89.25%	53.24%	92.41%	+3.17 pp
Condition Coverage	83.27%	45.75%	87.17%	+3.91 pp

ผลนี้แสดงว่าแม้ UTBot-only coverage ต่ำกว่า developer tests แต่ test ที่สร้างมีส่วนครอบคลุม code paths ที่ developer tests ยังไม่ครอบคลุม เมื่อรวมกัน line coverage เพิ่ม 3.17 percentage points และ condition coverage เพิ่ม 3.91 percentage points จึงสะท้อนประโยชน์ในฐานะ supplementary test generator มากกว่าการแทน developer tests ทั้งหมด

3.5 Performance

ผล raw generation มี 1,073 class-level rows ใช้เวลาสร้างรวมประมาณ 155,621.04 วินาที หรือ 43.23 ชั่วโมง และสร้าง test methods รวมประมาณ 154,416 รายการ คำนี้นี้เป็นภาระด้าน compute ที่สูง และจำนวน test ที่มากไม่ได้แปลว่า fault detection จะสูงโดยอัตโนมัติ เพราะความเข้ากันได้กับ dependency/JUnit/build ของโครงการเก่ายังเป็นคอขวดสำคัญ

4. การทดลองด้วย TARDIS

4.1 Pipeline

TARDIS ใช้แนวทาง Concolic/Symbolic Execution และพึ่งพา Docker, Z3/JBSE และ EvoSuite ใน pipeline ของกลุ่ม สคริปต์ `run_tardis.py` รองรับ auto-resume และ fallback จากระดับ class ลงสู่ระดับ method เมื่อ class-level generation หมดเวลา ส่วน `measure_coverage.py` ทำหน้าที่ merge tests, แก้ shading, วัด coverage และประเมิน fault detection

4.2 Configuration

ตารางที่ 5 Configuration ที่ใช้ในการทดลอง TARDIS

Configuration	Value
CLASS_TIMEOUT	120 seconds
FALLBACK_METHOD_TIMEOUT_CAP	60 seconds
MAX_WORKERS	2
Environment	Linux/WSL2 + Docker
Recommended RAM	8-16 GB (README แนะนำ 10-12 GB สำหรับ WSL2)

4.3 ผลที่บันทึก

ไฟล์ `Tardis/result_csv/Coverage_Result_All.csv` มี 854 rows ครอบคลุม 854 bugs โดยวัด line coverage สำเร็จ 549 bugs และบันทึก `Fault_Detected=YES` จำนวน 16 bugs หรือ 1.87% ของ 854 bugs ค่า line coverage เฉลี่ยใน 549 bugs ที่วัดได้เท่ากับ 15.59% และค่า branch coverage เฉลี่ยเท่ากับ 6.79% หากคำนวณแบบ weighted จาก covered/total elements จะได้ weighted line coverage 7.85% และ weighted branch coverage 3.27%

จากไฟล์ Coverage_Result_All.csv พบว่า TARDIS มี Eval_Status = OK จำนวน 545 bugs, NO_TESTS จำนวน 298 bugs และ compile/run fail รวม 11 bugs ขณะที่ Runner_Status แสดงข้อจำกัดด้าน environment อย่างชัดเจน เช่น DOCKER_FAIL, KILLED_WITH_TESTS, TIMEOUT, SUCCESS_ON_TIMEOUT และ fallback states. สิ่งนี้ทำให้ TARDIS เป็นตัวอย่างสำคัญของ algorithm ที่มีศักยภาพเชิงทฤษฎีอาจมี operational friction สูงเมื่อนำมารันกับ benchmark ขนาดใหญ่

4.4 ผลรายโครงการ

ตารางที่ 6 ผลการทดลอง TARDIS แยกรายโครงการ

Project	Bugs	Coverage measured	Recorded detected	Rate	Avg line	Avg branch
Chart	26	12	1	3.85%	23.88%	2.92%
Cli	39	29	0	0.00%	3.93%	2.43%
Closure	174	98	2	1.15%	8.07%	2.30%
Codec	18	16	0	0.00%	29.00%	23.39%
Collections	28	25	0	0.00%	17.52%	10.00%
Compress	47	47	2	4.26%	11.94%	5.09%
Csv	16	15	1	6.25%	27.49%	11.13%
Gson	18	13	1	5.56%	12.57%	2.64%
JacksonCore	26	19	2	7.69%	12.03%	4.87%
JacksonDataBind	110	49	0	0.00%	16.40%	4.42%
JacksonXml	6	2	1	16.67%	3.70%	0.85%
Jsoup	93	61	1	1.08%	16.81%	6.00%
JXPath	22	10	0	0.00%	20.60%	8.65%
Lang	61	48	2	3.28%	18.01%	10.05%
Math	106	69	3	2.83%	21.12%	12.16%
Mockito	38	25	0	0.00%	22.62%	6.98%
Time	26	11	0	0.00%	15.32%	6.69%

5. การทดลองด้วย Claude Sonnet 5

5.1 Prompt และ configuration

Claude pipeline ใช้ provider ของมหาวิทยาลัย (`gen.ai.kku.ac.th`) และ model id `claude-sonnet-5` โดยตั้ง temperature = 0, workers = 2 และ classpath context แบบชื่อ JAR (`cp\_test\_mode = names`). Prompt กำหนดบทบาทเป็น Java testing engineer ให้สร้าง JUnit 4, เน้น branch coverage, boundary/null/empty/invalid input, ห้ามเดา behavior ที่ไม่มีใน source และให้สรุป branch/condition ที่ test method แต่ละตัวครอบคลุม

Pipeline checkout buggy version, อ่าน `classes.modified`, สร้าง prompt รายการส, เรียก AI, ดึง Java code จากคำตอบ, compile และวัด coverage จากนั้นเปรียบเทียบ failing tests ระหว่าง buggy และ fixed เพื่อกำหนด `kills\_bug`

5.2 ผลรวม

ตารางที่ 7 ผลการทดลองโดยรวมของ Claude Sonnet 5

Metric	Result
Bugs represented in summary	849
Target classes	1,055
Compile rate	73.7%
Bugs killed/detected	46
Fault Detection Rate	5.4%
Average Line Coverage	81.14%

5.3 ผลรายโครงการ

ตารางที่ 8 ผลการทดลอง Claude Sonnet 5 แยกรายโครงการ

Project	Bugs	Classes	Compile	Killed	FDR	Avg line
Chart	26	28	85.7%	9	34.6%	92.90%
Cli	39	52	84.6%	0	0.0%	14.42%
Closure	173	220	51.8%	3	1.7%	66.57%
Codec	15	18	88.9%	2	13.3%	98.11%

ตารางที่ 8 ผลการทดลอง Claude Sonnet 5 แยกรายโครงการ (ต่อ)

Collections	28	28	89.3%	4	14.3%	95.74%
Compress	47	58	87.9%	3	6.4%	77.80%
Csv	16	17	76.5%	1	6.2%	99.02%
Gson	18	21	71.4%	0	0.0%	88.78%
JacksonCore	26	35	85.7%	4	15.4%	75.98%
JacksonDatabind	110	157	53.5%	1	0.9%	75.26%
JacksonXml	6	6	100.0%	0	0.0%	54.97%
Jsoup	92	123	86.2%	6	6.5%	94.93%
JXPath	22	35	65.7%	1	4.5%	77.39%
Lang	61	61	91.8%	2	3.3%	94.62%
Math	106	119	93.3%	8	7.5%	92.67%
Mockito	38	46	73.9%	1	2.6%	96.97%
Time	26	31	83.9%	1	3.8%	88.35%

6. การทดลองด้วย Gemini

6.1 Prompt และ configuration

Gemini ใช้ prompt ที่กำหนดบทบาทเป็น Senior Java Test Automation Engineer และระบุให้สร้าง JUnit 4 โดยมุ่ง branch/condition coverage รวมทั้ง boundary limits, null/empty values และ invalid states. Source code ของ target class และ classpath ถูกแทรกใน prompt ก่อนส่งให้โมเดล จากนั้น pipeline compile test, วัด coverage, รันบน buggy และ fixed version และบันทึก 'kills_bug'

6.2 ผลรวม

ตารางที่ 9 ผลการทดลองโดยรวมของ Gemini

Metric	Result
Active bugs	854 / 854
Bugs with ≥ 1 compilable generated test	366 (42.86%)
Bugs detected (kills_bug)	20 / 854
All-bug FDR	2.34%
FDR among bugs with compilable tests	5.46%

ตารางที่ 9 ผลการทดลองโดยรวมของ Gemini (ต่อ)

Generated test files (class-level)	1,067
Class-level compile success	395 / 1,067 (37.0%)
Average Line Coverage	91.33%
Average Condition Coverage	85.41%

Gemini มี average coverage สูงในกลุ่มที่วัดได้ แต่ compile failure ยังคงเป็นข้อจำกัดสำคัญ และ README ระบุว่า coverage บางส่วน อาจรวม developer tests ที่เหลืออยู่เพราะชื่อ test file สามารถชนกับ test เดิมได้ ดังนั้น coverage ต้องตีความต่างจาก UTBot weighted UTBot-only coverage นอกจากนี้บางโครงการใช้ JUnit/classpath รุ่นเก่า ทำให้ไม่สามารถตัดสิน kills_bug ได้ทุกกรณีอย่างสมบูรณ์

บทที่ 4

ผลการทดลอง และการเปรียบเทียบ

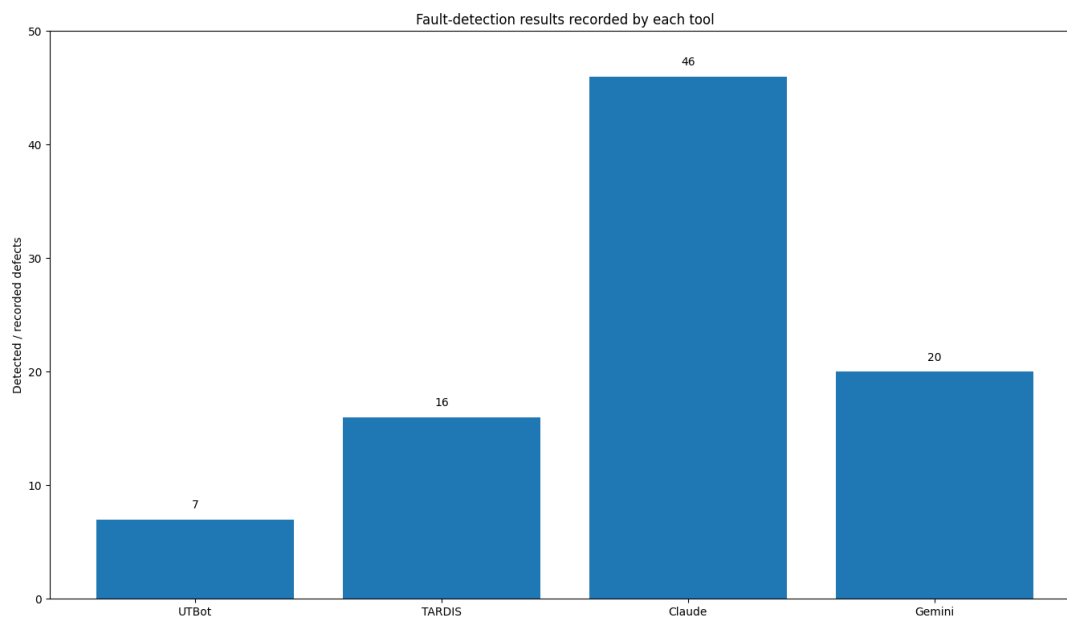
1. ภาพรวมผลลัพธ์

ตารางที่ 10 ภาพรวมผลการทดลองของเครื่องมือทั้ง 4 ตัว

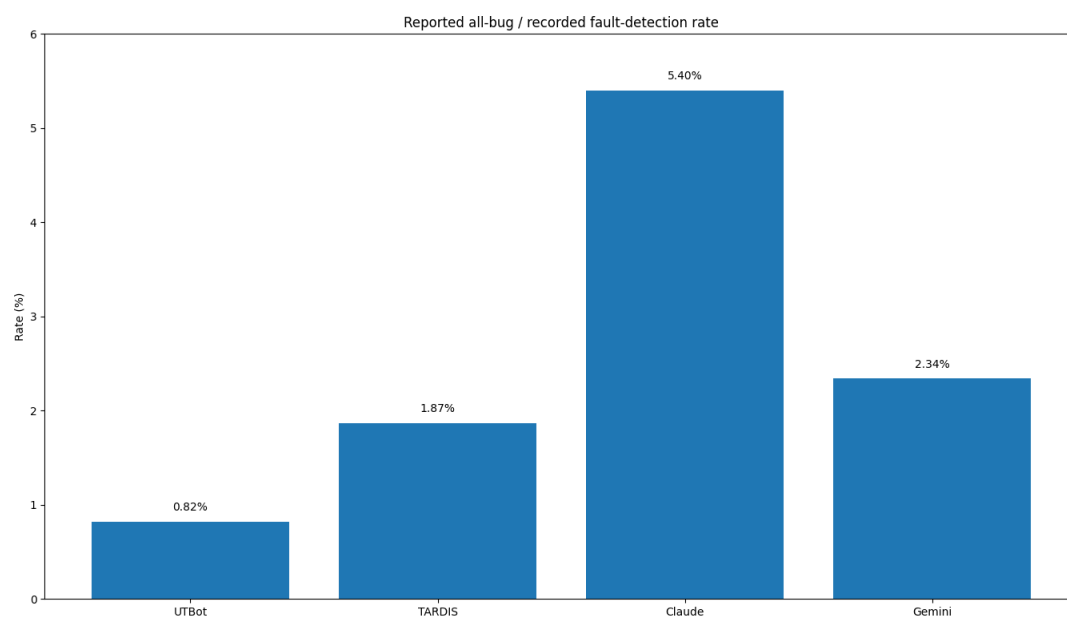
Tool	Type	Bugs	Executable / evaluable	Detected	Reported FDR	Line coverage	Condition/branch
UTBot	Automated	854	139 executable	7	0.82% (all) 5.04% (executable)	53.24% weighted (UTBot-only)	87.17% combined condition
TARDIS	Automated	854	549 measured coverage	16 recorded	1.87% recorded	15.59% avg. over measurable bugs	6.79% avg. branch over measurable bugs
Claude Sonnet 5	AI-assisted	849	1,055 target classes 73.7% compile rate	46	5.4%	81.14% avg. line coverage	Line metric configured
Gemini	AI-assisted	854	366 bugs with ≥ 1 compilable test	20	2.34% (all) 5.46% (compilable)	91.33% avg. line coverage	85.41% avg. condition coverage

ตารางนี้ใช้เพื่อสรุปข้อมูลที่แต่ละ pipeline รายงาน ไม่ใช่คะแนนเดียวกันทุกคอลัมน์ โดยเฉพาะ coverage: UTBot ใช้ weighted element counts, TARDIS ใช้ค่าเฉลี่ยเฉพาะ bugs ที่วัดได้, Claude/Gemini รายงาน average coverage ระดับ class/กลุ่มที่วัดได้

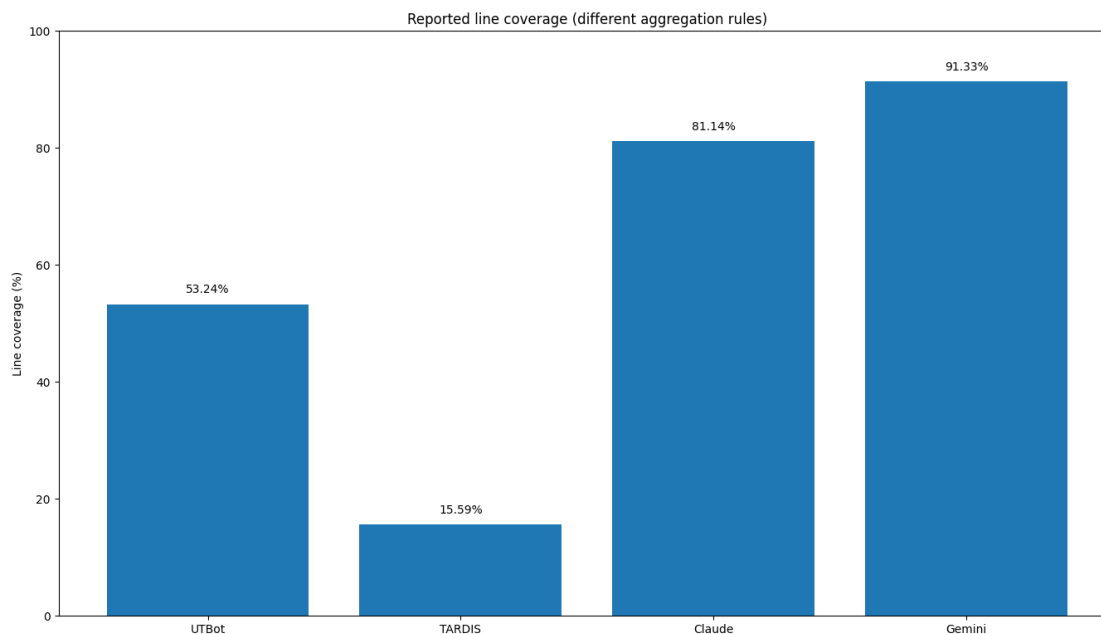
ดังนั้นตัวเลขเหล่านี้เหมาะกับการวิเคราะห์ภายในเครื่องมือและการมองแนวโน้ม มากกว่าการจัดอันดับตรง ๆ



ภาพที่ 2 จำนวน defect ที่แต่ละ pipeline บันทึกว่า detect/kill ได้



ภาพที่ 3 Fault-detection rate ตาม denominator ที่แต่ละ pipeline รายงาน



ภาพที่ 4 Line coverage ที่รายงาน

2. การตีความ Fault Detection

Claude มีจำนวน bugs killed มากที่สุดในข้อมูลของ AI pipeline (46/849) ขณะที่ Gemini ได้ 20/854 และ UTBot ได้ 7/854 สำหรับ TARDIS ค่า recorded Fault_Detected คือ 16/854 (1.87%) โดยมี coverage ที่วัดได้ 549 bugs จึงควรตีความแยกจากเกณฑ์ buggy-fail/fixed-pass ของเครื่องมืออื่น และไม่ควรรใช้ตัวเลขนี้เพียงค่าเดียวเพื่อสรุปว่าเครื่องมือใดดีที่สุด

3. การตีความ Coverage

Coverage สูงไม่ได้รับประกัน fault detection สูงเสมอไป ตัวอย่างเช่น Gemini มี average line coverage 91.33% แต่ all-bug FDR 2.34% ขณะที่ UTBot-only weighted line coverage 53.24% แต่เมื่อรวมกับ developer tests สามารถเพิ่ม line coverage ได้ 3.17 pp จึงชี้ว่าความหลากหลายของ paths ที่ test เพิ่มเข้ามาอาจมีคุณค่าแม้ coverage ของ generated tests เองไม่สูงที่สุด

บทที่ 5

การวิเคราะห์ผล และข้อจำกัด

1. Automated generation vs AI-assisted generation

Automated generators มีข้อได้เปรียบด้านการทำงานเชิงระบบและไม่ต้องพึ่ง natural-language reasoning แต่ต้องผูกกับ runtime/build environment อย่างแน่นหนา โดยเฉพาะ TARDIS ที่ต้องใช้ Docker, solver และ resource สูง และ UTBot ที่ต้องจัดการ Java/JUnit/dependency compatibility ของโครงการเก่า ขณะที่ AI-assisted tools สามารถอ่าน source แล้วสร้าง test ที่เจาะ boundary และ semantics ได้โดยตรง แต่มีความเสี่ยงเรื่อง API hallucination, import/classpath ผิด, test compile ไม่ผ่าน และผลที่ไม่ deterministic หาก configuration ไม่คงที่

2. Coverage กับ Fault Detection ไม่ใช่สิ่งเดียวกัน

Coverage วัดว่ามี code elements ถูก execute มากเพียงใด แต่ไม่ได้บอกว่า assertion สามารถแยก behavior ที่ผิดจาก behavior ที่ถูกต้องได้หรือไม่ การตรวจบักต้องมี oracle ที่มีคุณภาพ ซึ่งในโครงงานนี้ operationalize ผ่านการ fail บน buggy และ pass บน fixed. ดังนั้น test ที่วิ่งผ่านหลาย branch แต่ assertion อ่อนอาจมี coverage สูงแต่ไม่ kill bug ในทางกลับกัน test ที่เจาะจุด defect โดยตรง อาจ kill bug ได้แม้ coverage โดยรวมไม่สูง

3. ความสำคัญของ Executability

ผล UTBot และ Gemini แสดงว่าการแยก all-bug FDR กับ FDR ในกลุ่มที่ test ใช้งานได้ช่วยให้เห็นสองปัญหาคนละครูป all-bug FDR สะท้อน end-to-end practicality ส่วน executable/compilable FDR สะท้อน effectiveness เมื่อผ่าน compatibility barrier แล้ว สำหรับ benchmark ที่มีโปรเจกต์เก่าหลากหลาย build system การรายงานเพียงค่าเดียวอาจปิดบังสาเหตุที่แท้จริงของความล้มเหลว

4. ความน่าเชื่อถือของ comparison

การเปรียบเทียบที่เชื่อถือได้ควรใช้ target bugs, timeout, environment, fault-detection definition และ coverage aggregation เดียวกันทั้งหมด ข้อมูลปัจจุบันยังมีความต่างบางส่วน เช่น Claude summary 849 bugs, TARDIS ใช้คอลัมน์ Fault_Detected ใน Coverage_Result_All.csv โดยตรง และ Gemini coverage มี

ข้อจำกัดเรื่อง developer tests ที่อาจยังอยู่ใน project ด้วยเหตุนี้รายงานจึงนำเสนอผลแบบ transparent และ ไม่บังคับให้ตัวเลขที่นิยามต่างกันกลายเป็น metric เดียวกัน

5. ปัญหา ข้อจำกัด และ Reproducibility

5.1 ปัญหาที่พบ

ตารางที่ 11 ปัญหาและข้อจำกัดที่พบในการทดลอง

Tool / Area	Observed limitations
UTBot / Defects4J on Windows	Perl module, Windows path conversion, patch application, Major/Ant wrapper, CRLF/LF, project-specific build scripts
TARDIS	Docker/WSL integration, Z3/JBSE resource usage, timeout, killed process, fallback method generation, shading/name collision
Claude	Compile failure บางคลาส, API quota/streaming, context size, dependency/classpath compatibility
Gemini	Compile failure, JUnit/classpath รันเก่า, timeout, zero-shot generation และ coverage ที่อาจรวม developer tests

5.2 Reproducibility artifacts

Repository จัดเก็บ code, prompt, configuration, generated tests และ result files แยกตามเครื่องมือ โดย UTBot มี version pin และ patch files สำหรับทั้ง Defects4J/UTBotJava, TARDIS มี runner/evaluator scripts และ CSV, Claude มี prompt/test/result/config, และ Gemini มี Code/Prompt/Result/TestCode.

ตารางที่ 12 Artifacts สำหรับการทำการทดลองของแต่ละเครื่องมือ

Tool	Key paths in repository
UTBot	UTBot/README.md; UTBot/scripts/; UTBot/patches/; UTBot/Result_Automated/
TARDIS	Tardis/README.md; Tardis/source_code/; Tardis/result_csv/; Tardis/data_analysis/
Claude	Claude-sonnet_5/run_ai_tests.py; Claude-sonnet_5/AI_Results/Prompt/; TestCode/; Result/; config_Claude.json
Gemini	Gemini/README.md; Gemini/Code/; Gemini/Prompt/; Gemini/Result/; Gemini/TestCode/

บทที่ 6

สรุปผล

โครงการนี้ดำเนินการตามเป้าหมายในการทดลอง Automatic Test Case Generation 2 ตัว (UTBot, TARDIS) และ AI-Assisting Tools 2 ตัว (Claude Sonnet 5, Gemini) กับ Defects4J พร้อมเก็บ generated tests, coverage, fault-detection evidence, configuration และ scripts ลงใน GitHub. ผลชี้ว่าทุกเครื่องมือสามารถสร้าง test ที่มีประโยชน์ได้ แต่ความสำเร็จ end-to-end ขึ้นกับ compatibility และ evaluator อย่างมากพอ ๆ กับ algorithm/model เอง

UTBot ให้หลักฐานชัดเจนว่าการใช้ generated tests เสริม developer tests สามารถเพิ่ม weighted line coverage 3.17 pp และ condition coverage 3.91 pp แม้ตรวจพบเพียง 7 bugs จาก dataset ทั้งหมด. AI tools ให้ coverage เฉลี่ยสูงในกลุ่มที่ compile/วัดได้ และ Claude/Gemini ตรวจพบ 46/849 และ 20/854 bugs ตามลำดับ แต่ยังเผชิญ compile/classpath compatibility. TARDIS แสดงความสามารถในการสร้าง tests จำนวนมากและวัด coverage ได้ 549 bugs แต่ตรวจพบตามคอลัมน์ Fault_Detected เพียง 16/854 bugs และ operational failures ของ Docker/timeout มีผลมากต่อผลลัพธ์

บทเรียนหลักคือไม่ควรเลือกเครื่องมือจาก coverage เพียงค่าเดียว การประเมิน automatic testing ที่เหมาะสมควรดู fault-detection oracle, executability, coverage gain, runtime/resource cost และ reproducibility ร่วมกัน และในการทดลองต่อไปควรทำ normalization ของ timeout, fault-detection criterion และ coverage aggregation ให้เหมือนกันทุกเครื่องมือเพื่อให้ comparison มีความยุติธรรมมากขึ้น